Machine Learning in Time Series

Lecture 4: Deep Learning for Sequences (RNN & LSTM)

Instructor: Marcelo Villena, PhD

Spring 2026

PhD Seminar in Econometrics / Data Science
Università degli Studi di Bari Aldo Moro

Agenda

1. Seminar Overview & Roadmap
2. Mathematical Foundations: FFNNs & The Chain Rule
3. The Genesis of Sequence Models: RNN & LSTM
4. The Anatomy of an LSTM: Mathematical Mechanics
5. Comparative Synthesis
6. Readings Discussion
7. Lecture 4 Mini-Lab: LSTM Forecasting
8. Concluding Remarks

Seminar Overview & Roadmap

The 6-Lecture Journey

Where we are

In Lecture 3, we used BVARs and Sparse VARs to model high-dimensional linear dynamics. Today, we relax linear functional-form restrictions and allow nonlinear sequential dynamics to explore how Deep Learning architectures capture complex memory.

Lecture 1: Historical Evolution, the Prediction Problem, and Shrinkage.

Lecture 2: Kernel Methods and Support Vector Machines (SVM).

Lecture 3: High-Dimensional Linear Dynamics (BVAR, Sparse VAR).

Lecture 4: Deep Learning for Sequences (RNN/LSTM).

Lecture 5: Trees, Boosting, and Ensembles.

Lecture 6: NLP, Attention, and Causal ML.

Mathematical Foundations: FFNNs & The Chain Rule

The Anatomy of a Feed-Forward Neural Network (FFNN)

Before discussing sequences, we must understand the base architecture. A Neural Network is a series of layers containing "neurons."

The Logic of a Single Neuron:

1. **Input (X):** Economic data (e.g., Inflation).
2. **Weights (W):** The parameters the model learns to quantify the importance of the input.
3. **Bias (b):** An intercept term to shift the function.
4. **Activation (f):** A non-linear squashing function (like Sigmoid or Tanh) applied to the sum.

Hidden Layer Equation: $h = f(b_h + W \cdot X)$
Output Forecast Equation: $O = f_o(b_o + V \cdot h)$

Numerical Example: The Forward Pass

Let's trace how an economic input physically becomes a forecast in a simple two-layer network.

The Setup:

- Input ($X = 2$): Suppose unemployment is currently 2%.
- Weights: $W = 0.5$, $V = -1$.
- Biases: $b_h = 0, b_o = 0$.
- Activation: Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$.

The Forward Pass Calculation:

1. **Hidden Neuron Input:** $z_h = (W \cdot X) + b_h = (0.5 \cdot 2) + 0 = 1$.
2. **Hidden Neuron Output (Activation):** $h = \sigma(1) = \frac{1}{1+e^{-1}} \approx 0.731$.
3. **Final Forecast Input:** $z_o = (V \cdot h) + b_o = (-1 \cdot 0.731) + 0 = -0.731$.
4. **Final Forecast Output:** $O = \sigma(-0.731) = \frac{1}{1+e^{0.731}} \approx 0.324$.

Neural Networks as Compositions of Functions

How does the network "learn" the optimal weights (W, V) to make the forecast accurate? We must treat the neural network as a deeply nested mathematical function.

In a standard network, a hidden unit h_j computes a weighted sum and applies a non-linear activation:

$$h_j = f_j\left(b_j + \sum_i w_{ij} x_i\right)$$

The output layer o then transforms those hidden states:

$$o = f_0\left(b_o + \sum_j w_{jo} h_j\right)$$

Key idea: The network is a deep composition: $o = f_0(f_1(\ldots(X)))$.

The Chain Rule

Because the network is a composition of functions, finding the optimal weights requires us to measure how a tiny change in a weight deep inside the network affects the final forecast error.

The Chain Rule

If a function is a composition $y = f(g(x))$, then the derivative is:

$$\frac{dy}{dx} = f'(g(x)) \cdot g'(x)$$

Intuition: A small change in x affects $g(x)$, and that change propagates to $f(g(x))$. The overall effect is the product of both effects. Differentiate the outside, keep the inside; multiply by the inside derivative.

Backpropagation: The Chain Rule in Action

To train the network, we minimize a loss function (e.g., MSE):

$$S^2 = \sum_{t=1}^{T} (y_t - o_t)^2$$

To update a specific weight w early in the network, we apply the chain rule backwards from the output:

$$\frac{\partial S^2}{\partial w} = \frac{\partial S^2}{\partial o} \cdot \frac{\partial o}{\partial h_j} \cdot \frac{\partial h_j}{\partial w}$$

Backpropagation Algorithm:

1. Compute derivatives layer by layer (from output back to input).
2. Update weights using gradient descent: $w \leftarrow w - \eta \frac{\partial S^2}{\partial w}$.

Without the chain rule, neural networks cannot learn.

The Genesis of Sequence Models: RNN & LSTM

The Failure of Standard FFNNs for Time Series

The Econometric Problem

If we feed an FFNN an AR(12) vector of inflation, it treats lag 1 and lag 12 as entirely independent spatial features. It has no concept of **temporal order** or **sequential flow**. Furthermore, it cannot handle sequences of variable lengths.

The Genesis of Recurrent Neural Networks - RNN (1980s): Researchers realized that to process time, a network needs an internal "memory" or "state" that persists across time steps. Instead of just passing information forward to the next layer, an RNN passes information *laterally* to its own future self.

Recurrent Neural Networks (RNN): Pros and Cons

At time t, the RNN receives the current data X_t AND the hidden state from the previous time step h_{t-1}.

$$h_t = \tanh(WX_t + Uh_{t-1} + b)$$

The Pros:

- **Sequential Memory:** The state h_{t-1} contains a compressed summary of history.
- **Parameter Sharing:** The matrices W and U are exactly the same at every time step, drastically reducing the number of parameters compared to a massive VAR.

The Cons (The Fatal Flaw):

- **Vanishing Gradients:** Because the same matrix U is multiplied repeatedly across time during Backpropagation Through Time (BPTT), the signal shrinks or explodes.
- **Result:** If the largest singular value of U is < 1, gradients decay roughly like $\|U\|^t$; if > 1, they explode. Standard RNNs physically *cannot* remember long-term macro dependencies.

The Anatomy of an LSTM: Mathematical Mechanics

The Core Innovation: The Cell State

To solve the Vanishing Gradient problem, Hochreiter & Schmidhuber (1997) introduced the **Long Short-Term Memory (LSTM)** network.

The Cell State (C_t): Unlike a standard RNN which constantly overwrites its hidden state, an LSTM has a primary "conveyor belt" running straight down the entire chain. It allows information to flow across time with only minor linear interactions.

The Gates: The LSTM has the ability to carefully add or remove information to the cell state, regulated by structures called **gates**. Gates are composed of a sigmoid neural net layer (σ) and a pointwise multiplication operation ($*$). Because σ outputs values between 0 and 1, it dictates exactly *how much* of a signal should be let through.

LSTM Math Step 1: The Forget Gate

The first step is deciding what information from the past we are going to throw away.

The Forget Equation

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Econometric Intuition: The gate looks at the previous hidden state (h_{t-1}) and the current macroeconomic data (x_t). It outputs a number between 0 and 1 for each number in the cell state C_{t-1}.

- A '1' represents "keep this completely."
- A '0' represents "get rid of this completely."
- *Example:* If inflation undergoes a structural regime shift, the forget gate learns to discard the memory of the old regime.

LSTM Math Step 2: The Input Gate

The next step is deciding what new information we are going to store in the cell state. This has two parts:

The Input Equations

1. **Input Gate Layer** (decides which values we'll update):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

2. **Candidate Values** (creates a vector of new candidate values):

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Mechanics: The tanh layer creates new possible memory vectors (e.g., a recent shock to oil prices). The sigmoid input gate i_t decides exactly which components of that new shock are actually relevant enough to be written into the long-term memory.

LSTM Math Step 3: Updating the Cell State

Now we actually update the old cell state (C_{t-1}) into the new cell state (C_t).

The Update Equation (Solving the Vanishing Gradient)

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Why this is a breakthrough: Notice the **addition** ($+$) operator. In a standard RNN, the state is updated via endless matrix multiplication, which causes gradients to vanish or explode during backpropagation. By using addition to update the cell state, the LSTM provides an uninterrupted "gradient highway," allowing the model to flawlessly remember shocks from dozens of time steps ago!

LSTM Math Step 4: The Output Gate

Finally, we need to decide what we are going to output as our forecast (and pass to the next time step as the new hidden state).

The Output Equations

1. **Output Gate** (decides what part of the cell state to output):

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

2. **Final Hidden State** (push cell state through tanh and multiply):

$$h_t = o_t * \tanh(C_t)$$

Mechanics: We push the cell state values to be between -1 and 1 (using tanh) and multiply it by the output gate. We only broadcast the exact pieces of memory required to predict y_{t+1} at this specific moment.

Comparative Synthesis

Comparing Architectures for Econometrics

Feature	FFNN	RNN	LSTM
Data Flow	Strictly Feed-Forward	Recurrent (Loops)	Recurrent + Cell State Highway
Memory Type	None (Spatial only)	Short-Term Sequential	Long-Term & Short-Term Sequential
Gradient Behavior	Stable (for shallow nets)	**Vanishing / Exploding**	Stable (protected by addition)
Structural Complexity	Low (Simple weights)	Medium (Hidden states)	High (3 Gates + Cell State)
Best Macro Use Case	Static, cross-sectional mapping (e.g., Yield Curve at time t)	Highly persistent, short-horizon univariate series	Complex, multi-regime time series with long-range dependencies

Concluding the Deep Sequence Journey

- **The Evolution:** We moved from FFNNs (no time awareness) to RNNs (short-term time awareness) to LSTMs (gated, long-term time awareness).
- **The Mathematical Fix:** The LSTM's cell state replaces continuous matrix multiplication with linear addition, elegantly solving the Vanishing Gradient problem that plagued standard RNNs.
- **The Econometric Trade-off:** The extreme flexibility of an LSTM comes at the cost of vast parameter complexity. While it can map non-linear macroeconomic dynamics better than a VAR, it is highly prone to overfitting on small datasets ($T < 500$) and does not deliver classical structural IRFs without additional identification/interpretability machinery.

Readings Discussion

Reading: Farrell, Liang, & Misra (2021)

Deep Neural Networks for Estimation and Inference

Deep Learning is notoriously a "Black Box." It gives us $\hat{y}$ (the pure prediction objective), but economists need $\hat{\beta}$ and p-values to conduct policy inference.

Where Inference Enters:

- Farrell et al. prove that if an NN is wide/deep enough, it attains non-parametric convergence rates.
- **The Necessity of Orthogonalization:** You cannot simply extract a "coefficient" from an LSTM. To do structural inference, we must use the NN purely as a nuisance parameter estimator (to flexibly model the conditional mean). We then use Neyman-orthogonal moments / debiasing logic to extract unbiased causal parameters.

Reading: Lim et al. (2021) - TFT

Temporal Fusion Transformers (TFT)

LSTMs are powerful, but when doing multi-horizon forecasting ($h = 1 \ldots 12$), how do we know *which* specific past lag caused the prediction at a specific horizon?

Attention & Training Stability:

- The TFT combines LSTM sequence processing with **Attention Mechanisms** to address multi-horizon forecasting.
- Attention acts as a data-driven weighting scheme. The algorithm learns to "look back" at specific points in time (e.g., looking exactly 12 months back for seasonal effects) and assigns them higher weights.
- **Training Stability:** The TFT uses specialized gating mechanisms that allow it to bypass unnecessary non-linear transformations, vastly improving training stability compared to standard LSTMs on noisy financial datasets.

Economic Interpretability Toolkit for LSTM

We acknowledged that LSTMs do not yield classical structural IRFs natively. How do we turn this "black box" into a usable economic object?

The Modern Econometrician's Toolkit:

- **Scenario/Impulse Experiments:** Shock the input path (e.g., artificially bump the Fed Funds rate at t) and trace the predicted forecast path vs. a baseline.
- **Local Linear Surrogates:** Approximate the complex LSTM around a specific date using a Taylor expansion or local linear model to extract temporary "coefficients."
- **Feature Attributions (SHAP / Attention):** Use Shapley values or Attention weights (like in TFT) to quantify which lags drove the current forecast *(Warning: these are predictive correlations, not structural causes)*.
- **Stability Checks:** Conduct rigorous rolling retraining to see if the network's behavior changes across macroeconomic regimes.

Lecture 4 Mini-Lab: LSTM Forecasting

Mini-Lab: The Project Goes Deep

Task

Continue the spine project (forecasting U.S. YoY core CPI). We keep the **same information set** (AR(12)-style lags) and compare a classical benchmark (OLS) against neural competitors (FFNN, RNN, LSTM) under a **leakage-robust** protocol.

Pipeline requirements (R `keras`):

1. **Scaling (train-only):** Standardize inputs using *training* moments (z-score) or min–max scaling. This improves numerical stability for tanh/sigmoid gates and makes optimization behave.
2. **3D tensors for sequences:** Convert the AR(12) lag matrix into `[samples, time_steps, features]` for RNN/LSTM.
3. **Architecture (minimal):** A single sequence layer (RNN or LSTM) + a Dense output layer; use early stopping.
4. **Evaluation:** Report leakage-robust OOS RMSE and compare against OLS (and optionally ridge) under the *same* split and preprocessing.

Understanding the 3D Tensor Shape

Unlike OLS/Ridge (2D design matrix), an LSTM consumes *windows of time*.

Keras 3D input tensor for AR(12) CPI

With T total observations and look-back $L = 12$:

- **samples** $= T - L$: number of rolling windows.
- **time_steps** $= L$: lags inside each window.
- **features** $= \mathbf{1}$: univariate CPI at each step (multivariate $\Rightarrow$ features = number of series).

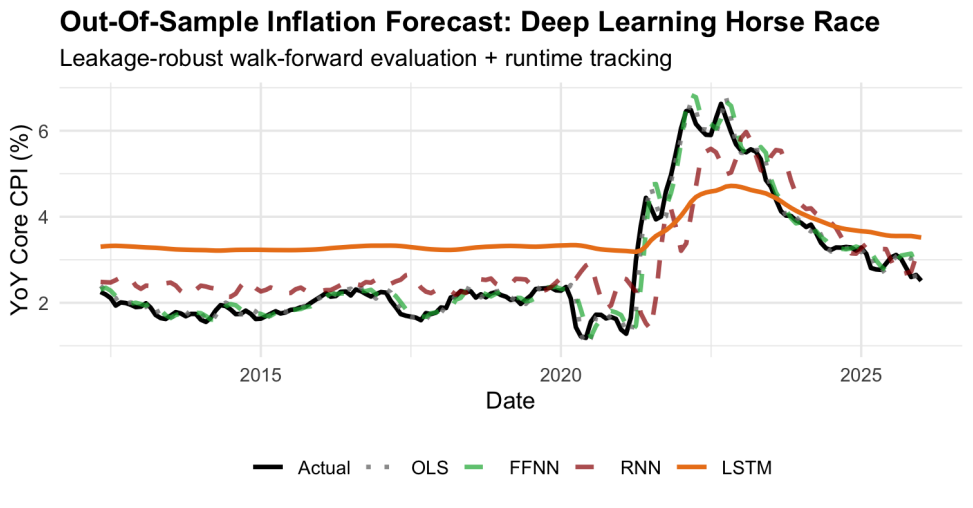
Debug tip: check `dim(X_train)`*. For CPI-only AR(12), Keras expects* `[samples, 12, 1]`*.*

Leakage-Robust LSTM Checklist

Deep learning comparisons fail most often due to subtle leakage or misaligned windows. Follow this checklist:

1. **Scaler fit on train only:** compute $(\bar{\pi}_{\text{train}}, s_{\text{train}})$ using training data; apply unchanged to test.
2. **Proper lag initialization:** the first test forecast must use the last 12 *training* lags (true information set at the split).
3. **No peeking in window construction:** ensure y_t is never included inside the input window used to predict y_t.
4. **Time-ordered validation:** early stopping must validate on a chronological end-block; set `shuffle=FALSE`.
5. **Sanity baselines:** compare to OLS AR(12) (and ridge if desired). If the NN cannot beat a simple baseline, interpret results cautiously.

OOS Forecast Horse Race (YoY Core CPI)



Protocol (fair + no leakage):

Accuracy vs Compute (OOS): Results and Interpretation

OOS RMSE + runtime (seconds)

Model	RMSE	Fit	Pred	Tot
OLS AR(12)	0.1854	0.001	0.000	0.001
FFNN	0.2755	8.414	0.052	8.466
RNN	0.8054	1.515	0.133	1.648
LSTM	1.2482	1.695	0.152	1.847

How to read this

- **OLS dominates** here: low-dimensional, near-linear dynamics + small macro sample.
- Neural nets add compute and can underperform without richer nonlinear signal or richer predictors.
- This is a *result*, not a failure: the protocol is designed so the comparison is credible.

When deep learning is more likely to help

- data-rich predictors (many series, text/news),
- regime shifts/nonlinearities, interactions,
- longer memory beyond short AR windows.

Concluding Remarks

Project: 10-Minute Check-In

Our Standard Closing Questions

1. **What changed in OOS RMSE?**
2. **Which failure mode did we fix (or create)?**

Today's Expected Answers:

- **RMSE Shift:** For a simple, short univariate series like our AR(12) inflation, the LSTM might actually *underperform* Ridge regression. Why? Deep learning requires massive data.
- **Failure Mode Fixed:** We no longer rely on rigid linear assumptions. If inflation dynamics exhibit nonlinear state dependence, an LSTM can represent it; whether it improves OOS accuracy depends on sample size, stability, and tuning.
- **Failure Mode Created: The Data Hunger.** LSTMs easily overfit small macroeconomic datasets ($T = 300$ months) and require extreme care with early stopping, dropout, and learning rate tuning.

Looking Ahead: Lecture 5

Next Session: Trees, Boosting, and Ensembles

Deep Learning is often overkill for standard tabular macro data. In Class 5, we look at the most dominant algorithms for non-linear, tabular forecasting: Decision Trees and Gradient Boosting.

Topics to prepare for:

- CART intuition and splitting logic.
- Random Forests (Bagging) to reduce variance.
- Gradient Boosting (XGBoost) and learning from residuals.

Reminder: Reading memos for Breiman (2001) and Goulet Coulombe et al. (2022) are due before the next Lecture!